

PAC-MAN GAME

A fully playable Pac-Man built from scratch in Java — no database, just Swing and pure OOP.

● Week 6
Complete

Java JDK 21

VS Code

Swing / JPanel

4 Unique Ghost AIs

WHAT WAS BUILT

Ghost Subclasses

- ✓ RedGhost.java — targets Pac-Man's exact tile (direct chase)
- ✓ BrownGhost.java — targets 4 tiles ahead of Pac-Man (ambush)
- ✓ BlueGhost.java — targets a point projected off RedGhost's position
- ✓ YellowGhost.java — chases from afar, flees to a corner up close
- ✓ All four extend Ghost.java and override `getTargetTile()` only

Targeting AI Engine

- ✓ `getTargetTile()` added to Ghost.java as an overridable hook
- ✓ `chooseDirection()` now picks the non-reversing move closest to target
- ✓ Greedy tile-distance comparison — no heavy pathfinding needed
- ✓ Dead-end fallback still allows reversal, same as Week 5
- ✓ Ghosts spawn in the open tunnel row to avoid wall overlap

KEY TECHNIQUES

`getTargetTile()` — One Hook, Four Personalities

`Ghost.java` declares a single protected method, `getTargetTile()`, that returns the tile a ghost is currently trying to reach. The base class implementation just returns Pac-Man's tile. Each subclass overrides only this one method — movement, wall collision, and rendering are inherited unchanged. This is the payoff of the Week 5 inheritance-ready design.

Greedy Distance-to-Target Movement

Each time a ghost reaches a tile center, `chooseDirection()` checks every non-reversing direction, simulates one tile-step in that direction, and measures straight-line distance to the current target tile. The direction that lands closest wins. This mirrors the original arcade ghosts' approach: no search algorithm, just local greedy comparison — simple, fast, and gives each ghost a distinct feel.

Four Distinct Targeting Strategies

RedGhost chases Pac-Man's live tile directly. BrownGhost aims 4 tiles ahead of Pac-Man's current direction, ambushing corners. BlueGhost doubles the vector from RedGhost through a point ahead of Pac-Man, producing unpredictable flanking paths. YellowGhost chases normally beyond 8 tiles away but retreats to a scatter corner once Pac-Man gets close, preventing it from ever fully cornering the player.

SOURCE CODE

Ghost.java (excerpt)

java

```
public class Ghost {
protected int row, col;
protected double x, y;
protected int dx, dy;
protected double speed = 1.5;
protected Maze maze;
protected PacMan pacman;
protected java.util.Random random = new java.util.Random();

// Default target: chase Pac-Man directly.
// Subclasses override this for their own personality.
protected int[] getTargetTile() {
return new int[] { pacman.getRow(), pacman.getCol() };
}

protected void chooseDirection() {
int[][] directions = {{0,-1},{0,1},{-1,0},{1,0}};
int[] target = getTargetTile();
int bestDx = dx, bestDy = dy;
double bestDist = Double.MAX_VALUE;
boolean found = false;

for (int[] d : directions) {
if (d[0] == -dx && d[1] == -dy) continue;
if (!canMove(d[0], d[1])) continue;
int nr = row + d[1], nc = col + d[0];
double dist = distance(nr, nc, target[0], target[1]);
if (dist < bestDist) {
bestDist = dist; bestDx = d[0]; bestDy = d[1]; found = true;
}
}
if (found) { dx = bestDx; dy = bestDy; }
}
// ...canMove(), isAligned(), draw() unchanged from Week 5
}
```

RedGhost.java

java

```
public class RedGhost extends Ghost {
public RedGhost(Maze maze, int startRow, int startCol, PacMan pacman) {
super(maze, startRow, startCol, "assets/sprites/RedGhost.png", pacman);
}

@Override
protected int[] getTargetTile() {
return new int[] { pacman.getRow(), pacman.getCol() };
}
}
```

BrownGhost.java

java

```

public class BrownGhost extends Ghost {
    public BrownGhost(Maze maze, int startRow, int startCol, PacMan pacman) {
        super(maze, startRow, startCol, "assets/sprites/BrownGhost.png", pacman);
    }

    @Override
    protected int[] getTargetTile() {
        int targetRow = pacman.getRow() + pacman.getDy() * 4;
        int targetCol = pacman.getCol() + pacman.getDx() * 4;
        return new int[] { targetRow, targetCol };
    }
}

```

BlueGhost.java

java

```

public class BlueGhost extends Ghost {
    private RedGhost redGhost;

    public BlueGhost(Maze maze, int startRow, int startCol, PacMan pacman, RedGhost redGhost) {
        super(maze, startRow, startCol, "assets/sprites/BlueGhost.png", pacman);
        this.redGhost = redGhost;
    }

    @Override
    protected int[] getTargetTile() {
        int aheadRow = pacman.getRow() + pacman.getDy() * 2;
        int aheadCol = pacman.getCol() + pacman.getDx() * 2;

        int targetRow = aheadRow + (aheadRow - redGhost.getRow());
        int targetCol = aheadCol + (aheadCol - redGhost.getCol());

        return new int[] { targetRow, targetCol };
    }
}

```

YellowGhost.java

java

```

public class YellowGhost extends Ghost {
    private final int scatterRow = Maze.ROWS - 2;
    private final int scatterCol = 1;

    public YellowGhost(Maze maze, int startRow, int startCol, PacMan pacman) {
        super(maze, startRow, startCol, "assets/sprites/YellowGhost.png", pacman);
    }

    @Override
    protected int[] getTargetTile() {
        int dr = row - pacman.getRow();
        int dc = col - pacman.getCol();
        double dist = Math.sqrt(dr * dr + dc * dc);

        if (dist > 8) {
            return new int[] { pacman.getRow(), pacman.getCol() };
        } else {
            return new int[] { scatterRow, scatterCol };
        }
    }
}

```

Ghost.java (full)

java

```

import java.awt.*;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.File;
import java.util.Random;

public class Ghost {
    protected int row, col;
    protected double x, y;
    protected int dx, dy;
    protected double speed = 1.5;
    protected BufferedImage sprite;
    protected Maze maze;
    protected PacMan pacman;
    protected Random random = new Random();

    public Ghost(Maze maze, int startRow, int startCol, String spritePath, PacMan pacman) {
        this.maze = maze;
        this.row = startRow;
        this.col = startCol;
        this.x = col * Maze.TILE_SIZE;
        this.y = row * Maze.TILE_SIZE;
        this.dx = 0;
        this.dy = 0;
        this.pacman = pacman;
        try {
            sprite = ImageIO.read(new File(spritePath));
        } catch (Exception e) {
            System.out.println("Could not load sprite: " + spritePath);
        }
    }

    public void update() {
        if (isAligned()) chooseDirection();
        if (canMove(dx, dy)) {
            x += dx * speed;
            y += dy * speed;
        } else {
            x = col * Maze.TILE_SIZE;
            y = row * Maze.TILE_SIZE;
            dx = 0; dy = 0;
        }
        row = (int) Math.round(y / Maze.TILE_SIZE);
        col = (int) Math.round(x / Maze.TILE_SIZE);

        if (x < -Maze.TILE_SIZE) x = Maze.COLS * Maze.TILE_SIZE;
        if (x > Maze.COLS * Maze.TILE_SIZE) x = -Maze.TILE_SIZE;
    }

    // Default target: chase Pac-Man directly.
    // Subclasses override this to give each ghost its own personality.
    protected int[] getTargetTile() {
        return new int[] { pacman.getRow(), pacman.getCol() };
    }

    protected void chooseDirection() {
        int[][] directions = {{0,-1},{0,1},{-1,0},{1,0}};
        int[] target = getTargetTile();

        int bestDx = dx, bestDy = dy;

```

```

double bestDist = Double.MAX_VALUE;
boolean found = false;

// Try all non-reversing directions, pick the one closest to target
for (int[] d : directions) {
    if (d[0] == -dx && d[1] == -dy) continue;
    if (!canMove(d[0], d[1])) continue;

    int nextRow = row + d[1];
    int nextCol = col + d[0];
    double dist = distance(nextRow, nextCol, target[0], target[1]);

    if (dist < bestDist) {
        bestDist = dist;
        bestDx = d[0];
        bestDy = d[1];
        found = true;
    }
}

// Dead end: allow reversal as a fallback
if (!found) {
    for (int[] d : directions) {
        if (canMove(d[0], d[1])) {
            int nextRow = row + d[1];
            int nextCol = col + d[0];
            double dist = distance(nextRow, nextCol, target[0], target[1]);
            if (dist < bestDist) {
                bestDist = dist;
                bestDx = d[0];
                bestDy = d[1];
                found = true;
            }
        }
    }
}

if (found) {
    dx = bestDx;
    dy = bestDy;
}

protected double distance(int r1, int c1, int r2, int c2) {
    int dr = r1 - r2;
    int dc = c1 - c2;
    return Math.sqrt(dr * dr + dc * dc);
}

protected boolean canMove(int testDx, int testDy) {
    if (testDx == 0 && testDy == 0) return false;
    double nextX = x + testDx * speed;
    double nextY = y + testDy * speed;
    int checkRow = (int) Math.round(nextY / Maze.TILE_SIZE);
    int checkCol = (int) Math.round(nextX / Maze.TILE_SIZE);
    return !maze.isWall(checkRow, checkCol);
}

protected boolean isAligned() {
    double diffX = x - col * Maze.TILE_SIZE;

```

```

double diffY = y - row * Maze.TILE_SIZE;
return Math.abs(diffX) < speed && Math.abs(diffY) < speed;
}

public void draw(Graphics g) {
    if (sprite != null) {
        g.drawImage(sprite, (int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE, null);
    } else {
        g.setColor(Color.RED);
        g.fillOval((int)x, (int)y, Maze.TILE_SIZE, Maze.TILE_SIZE);
    }
}

public int getRow() { return row; }
public int getCol() { return col; }
public double getX() { return x; }
public double getY() { return y; }
}

```

PacMan.java (addition)

java

```

// Add these getters to PacMan.java (needed by BrownGhost and BlueGhost
// to predict where Pac-Man is heading):

public int getDx() { return dx; }
public int getDy() { return dy; }

```

GamePanel.java (full)

java

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;

public class GamePanel extends JPanel implements Runnable {
    private Thread gameThread;
    private Maze maze;
    private PacMan pacman;
    private ScoreManager scoreManager;

    private RedGhost redGhost;
    private BrownGhost brownGhost;
    private BlueGhost blueGhost;
    private YellowGhost yellowGhost;

    private boolean gameOver = false;
    private boolean won = false;

    public GamePanel() {
        maze = new Maze();
        pacman = new PacMan(maze, 1, 1);
        scoreManager = new ScoreManager();

        // Spawn ghosts in the open tunnel row (row 10) to avoid wall overlap
        redGhost = new RedGhost(maze, 10, 9, pacman);
        brownGhost = new BrownGhost(maze, 10, 10, pacman);
        blueGhost = new BlueGhost(maze, 10, 8, pacman, redGhost);
        yellowGhost = new YellowGhost(maze, 10, 7, pacman);

        setBackground(Color.BLACK);
        setPreferredSize(new Dimension(
            Maze.COLS * Maze.TILE_SIZE,
            Maze.ROWS * Maze.TILE_SIZE + 40));
        setFocusable(true);

        addKeyListener(new KeyAdapter() {
            @Override
            public void keyPressed(KeyEvent e) {
                switch (e.getKeyCode()) {
                    case KeyEvent.VK_UP: pacman.setDirection(0, -1); break;
                    case KeyEvent.VK_DOWN: pacman.setDirection(0, 1); break;
                    case KeyEvent.VK_LEFT: pacman.setDirection(-1, 0); break;
                    case KeyEvent.VK_RIGHT: pacman.setDirection(1, 0); break;
                }
            }
        });

        gameThread = new Thread(this);
        gameThread.start();
    }

    @Override
    public void run() {
        while (true) {
            if (!gameOver && !won) {
                pacman.update();
                redGhost.update();
                brownGhost.update();
                blueGhost.update();
            }
        }
    }
}

```

```

yellowGhost.update();

int eaten = pacman.eatDot();
if (eaten == 2) scoreManager.addDot();
if (eaten == 3) scoreManager.addPowerPellet();

if (maze.countDots() == 0) won = true;
}
repaint();
try { Thread.sleep(16); }
catch (InterruptedException e) { e.printStackTrace(); }
}
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    // HUD - Score
    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.setColor(Color.WHITE);
    g2.drawString("SCORE", 10, 18);
    g2.setColor(Color.YELLOW);
    g2.drawString(String.valueOf(scoreManager.getScore()), 10, 34);

    // HUD - Lives
    g2.setColor(Color.WHITE);
    g2.drawString("LIVES", 180, 18);
    for (int i = 0; i < scoreManager.getLives(); i++) {
        g2.setColor(Color.YELLOW);
        g2.fillOval(180 + i * 22, 22, 14, 14);
    }

    // Title
    g2.setColor(Color.WHITE);
    g2.setFont(new Font("Arial", Font.BOLD, 16));
    g2.drawString("PAC-MAN", 85, 25);

    // Maze + characters
    Graphics2D g2maze = (Graphics2D) g2.create();
    g2maze.translate(0, 40);
    maze.draw(g2maze);
    pacman.draw(g2maze);
    redGhost.draw(g2maze);
    brownGhost.draw(g2maze);
    blueGhost.draw(g2maze);
    yellowGhost.draw(g2maze);
    g2maze.dispose();

    // Overlays
    if (won) drawOverlay(g2, "YOU WIN!", Color.YELLOW);
    else if (gameOver) drawOverlay(g2, "GAME OVER", Color.RED);
}

private void drawOverlay(Graphics2D g2, String message, Color color) {
    g2.setColor(new Color(0, 0, 0, 160));
    g2.fillRect(0, 0, getWidth(), getHeight());
    g2.setFont(new Font("Arial", Font.BOLD, 36));

```

```

g2.setColor(color);
FontMetrics fm = g2.getFontMetrics();
int x = (getWidth() - fm.stringWidth(message)) / 2;
int y = getHeight() / 2;
g2.drawString(message, x, y);
}
}

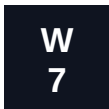
```

App.java, GameWindow.java, and Maze.java are unchanged from Week 5.

10-WEEK PROGRESS

Week 1 — Setup & Window	✓
Week 2 — Maze Design	✓
Week 3 — Pac-Man Character	✓
Week 4 — Dot Collection & Scoring	✓
Week 5 — Ghost Base Class	✓
Week 6 — All Four Custom Ghosts	✓
Week 7 — Power Pellets & Frightened Mode	0%
Weeks 8–10 — Polish + Ship	0%

UP NEXT



Power Pellets & Frightened Mode

Eating a power pellet should turn all four ghosts frightened: slow, blue-tinted, and edible for bonus points until they reset back to their spawn point.

GhostState enum · frightened timer · blue tint rendering · bonus scoring